
ADVANCED OPERATING SYSTEMS WITH UNIX

CH-2



What is Process Synchronization?

- In modern operating systems, multiple processes or threads often run **concurrently** (e.g., multitasking, multiprogramming, distributed systems).
- These processes may **share resources** such as:
 - CPU
 - Memory
 - Files
 - I/O devices
- **Process Synchronization** is the mechanism that ensures **safe and correct execution** of such processes so they do not interfere with each other and produce consistent results.

PROCESS SYNCHRONIZATION & COMMUNICATION (IPC, SEMAPHORES, MONITORS, MESSAGE PASSING)

Process Synchronization

- **Definition:** A mechanism that ensures multiple processes/threads execute in a way that prevents **race conditions** and guarantees **data consistency** when accessing shared resources.
 - Ensures that **concurrent processes/threads** execute in a controlled manner without interfering with each other.
 - Focuses on **critical section problem**.
 - **Problem:** When processes **concurrently** execute critical sections, results may vary → inconsistent data.
-

The Critical Section Problem

- **Definition**
 - A **critical section** is a portion of the program where a process **accesses shared resources** (like variables, data structures, files, or devices).
 - If more than one process enters the critical section at the same time → **Race Condition** occurs.
 - **Race condition** = final output depends on the **order of execution** of concurrent processes.
 - Example: Two people withdrawing money from the same bank account simultaneously → account balance may become inconsistent.
-

Critical Section Problem

- **Critical Section** = A code segment that accesses **shared data**.
- If multiple processes enter critical section at once → **race condition** occurs.

Requirements (Critical Section Problem)

1. **Mutual Exclusion** – Only one process executes critical section at a time.
2. **Progress** – If no process is in CS, a waiting process must be allowed to enter.
3. **Bounded Waiting** – A process must not wait indefinitely to enter CS.

Use Case: Updating a bank balance simultaneously by two ATMs.

ATM withdrawals from the same bank account (two people shouldn't withdraw simultaneously and cause inconsistency).

Conditions for a Correct Solution

- A solution to the critical section problem must satisfy **three essential requirements**:

1. Mutual Exclusion

1. At most one process can be inside the critical section at any given time.
2. Prevents race conditions.

2. Progress

1. If no process is executing in the critical section and some processes are waiting, then one of the waiting processes must be allowed to enter within a **finite amount of time**.

3. Bounded Waiting

1. After a process has made a request to enter its critical section, there exists a **limit** on the number of times other processes can enter before this request is granted.
 2. Prevents **starvation** (where one process waits indefinitely).
-

- **Hardware & Software Solutions**

- **A. Software Solutions**

- Peterson's Algorithm
- Dekker's Algorithm
- Bakery Algorithm (Lamport's algorithm for distributed systems)

- **B. Hardware Solutions**

- Modern hardware provides special instructions for synchronization:
 - Test-and-Set (TSL)
 - Compare-and-Swap (CAS)
 - Swap Instruction
-

- **Synchronization Tools (Provided by OS)**

To simplify process synchronization, OS provides **high-level constructs**:

1. **Semaphores** – Integer counters with wait() and Signal() operations
2. **Mutex Locks** – Binary locks for exclusive access
3. **Monitors** – High-level structure with condition variables.
4. **Message Passing** – Synchronization in distributed systems.

Real-World Examples of Process Synchronization

- **ATM Transaction**

- Two users withdrawing from the same account must be synchronized → otherwise negative balance may occur.

- **Printer Spooling**

- Multiple processes sending data to the printer → spooler must synchronize access to avoid mixing outputs.

- **Ticket Booking System**

- Multiple users booking the last seat → synchronization ensures only one gets it.

- **Database Management**

- Multiple users updating the same record in a database → must ensure consistency.

Issues Without Synchronization

If synchronization is not implemented properly:

- **Deadlock** – Two or more processes waiting forever for resources.
 - **Starvation** – Some processes never get CPU or resource time.
 - **Race Conditions** – Inconsistent or incorrect results.
-

2. Inter-Process Communication (IPC)

- Processes often need to **exchange data or coordinate actions**.
- IPC provides mechanisms for this.

Methods

1. Shared Memory

1. A common memory region is shared among processes.
2. **Faster**, since processes directly read/write memory.
3. Needs synchronization (e.g., semaphores).
4. **Use Case**: Databases, multiprocessing systems.

2. Message Passing

1. Processes communicate via **send/receive** primitives provided by OS.
 2. Easier in distributed systems.
 3. **Types**:
 1. Direct (explicit process name)
 2. Indirect (mailbox/queue)
 4. **Use Case**: Client-server systems, microkernels.
-

- **Inter-Process Communication (IPC) – Detailed Explanation**

- **1. Definition**

- In a multitasking or distributed system, multiple processes may need to **exchange data** and **coordinate actions**.
 - Since processes are isolated (especially in modern OS where each process has its own address space), they cannot directly access each other's memory.
 - **IPC provides mechanisms for processes to communicate and synchronize with each other.**
-

- **IPC Models A. Shared Memory**

- B. Message Passing**

- **A. Shared Memory**

- A region of memory is shared among processes.
 - Processes can directly **read/write** into this memory segment.
 - The **OS only sets up the shared memory region**, but processes are responsible for synchronization (using semaphores, mutexes, monitors).
 - **Features:**
 - **Fast** → direct access, no kernel intervention after setup.
 - **Efficient** for large volumes of data.
 - Requires explicit synchronization (risk of race conditions).
-

- **Message Passing**

-
- Processes communicate using **messages** (send/receive primitives).
 - The **OS provides the mechanism**, and ensures correctness.
 - Useful when processes **do not share memory** (e.g., in distributed systems).
 - **Features:**
 - Easier for communication across **different machines**.
 - Slower than shared memory (since messages go through kernel/communication layer).
 - Naturally synchronized (send/receive acts as a sync point).

- **Types of Message Passing:**

1. Direct Communication

1. Processes must explicitly name each other.
2. Example: send(P1, message) → sends directly to process P1.
3. Simple, but less flexible

2. Indirect Communication

- Messages are sent via **mailboxes (message queues)**.
 - don't need to know each other explicitly.
-
- Example: send(mailbox, message); receive(mailbox, message).
 - Flexible, supports multiple senders/receivers.

• 3. Semaphores

- Introduced by Edsger Dijkstra for synchronization.
- Integer variable used to control access to resources.

Types

- Binary Semaphore (Mutex) $\rightarrow$ Value = $\{0,1\}$. Used for mutual exclusion.
- Counting Semaphore $\rightarrow$ Value ≥ 0 . Used when multiple resource instances exist.

wait(S) (P operation): Decrement; if value < 0 , block process.

signal(S) (V operation): Increment; if value ≤ 0 , wake up a waiting process.

-
- **Use Case:** Producer-Consumer problem (buffer management).

Advantages:

- Simple, low-level synchronization.
- Can manage multiple instances of resources.

Disadvantages:

- Can cause **deadlock** if not handled properly.
 - May lead to **starvation** (some processes wait too long).
-

4. Monitors

A high-level synchronization construct that combines:

- *Shared variables*
- *Procedures (operations)*
- *Synchronization (condition variables)*

At most one process can be active inside a monitor at any time → ensures mutual exclusion automatically.

Condition Variables

- Used for process coordination inside monitors.

Operations:

- `wait()` → suspends process until signaled
 - `signal()` → resumes a waiting process.
-

Advantages:

- Easier to program than semaphores.
- Safer (prevents common programming mistakes).

Disadvantages:

- Requires language/OS support.
 - More complex to implement at kernel level.
-

5. Message Passing

- IPC mechanism where processes send and receive messages.
- Types

1. *Direct Communication*

- Processes must explicitly name each other.
- Example: send(P1, message), receive(P2, message)

2. *Indirect Communication*

- Uses mailboxes/ports. Processes communicate via queues.
 - Example: send(Mailbox1, message), receive(Mailbox1, message)
-

Synchronization

- **Blocking (Synchronous):** Sender waits until message delivered; receiver waits until message received.
- **Non-Blocking (Asynchronous):** Sender continues after sending, receiver may poll or get notification.

Use Case: Client-server communication, distributed systems.

- **Advantages:**
 - Easier to implement in distributed systems.
 - Provides strong process isolation.
 - **Disadvantages:**
 - Slower than shared memory.
 - Requires OS kernel overhead for message management.
-